VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science Engineering and Information Systems

BCSE302L Database Systems

Course Project

BOM-Track

A Database for Hardware Bills of Materials, Part Stock and Board Builds

Review I: Problem Analysis and Database Design

Submitted by	: Vishwak
Register Number	: 24BYB0053
Implementation	: PostgreSQL 16 (SQL, PL/pgSQL), Python
Faculty	:

October 2026

Contents

1. Introduction and Problem Statement
2. Objectives, Users and Scope
3. Requirement Analysis
4. ER Diagram
5. Relational Schema
6. Design Notes

1. Introduction and Problem Statement

Every PCB I design in KiCad ends with a bill of materials (BOM) exported as a CSV file. For my 8051-based handheld game board, uGameBox, that file has 28 lines. Each line groups several components under one value and footprint, for example "R1-R8,R10" for nine 1k resistors.

The CSV is fine as a parts list, but it cannot answer the questions that come up once I actually want to build the board:

- How much will one board cost, using current supplier prices?
- Which parts do I already have, and which ones do I still need to buy?
- If I want to build three boards, what is the shortfall for each part?
- Which parts are shared between my different boards, so I can buy them together?

Right now these answers are worked out by hand in a spreadsheet, and they go out of date as soon as a price changes or parts get used. The file also has some data problems. The Refs column stores a list of designators inside one cell. The Qty column is a count that can be computed from that list. The Value column is sometimes a real value ("10k") and sometimes just a label: the nine identical 6 mm push buttons appear as nine separate lines because they are named BTN0 to BTN7 and RESET.

BOM-Track stores this information in a relational database. Only the basic facts are stored (which part sits at which position on which board, what price a supplier quoted, how many parts came in or went out). Totals such as board cost, stock on hand, shortfall and build readiness are calculated by SQL views and PL/pgSQL functions whenever they are asked for, so they are never stale and never typed in by hand.

2. Objectives, Users and Scope

2.1 Objectives

- Import a KiCad BOM CSV into normalized tables, with one row per placed component.
- Keep a single catalogue of parts so that the same physical part is stored only once, even when it is used on many boards.
- Record supplier price quotes and stock movements as dated facts.
- Plan and record builds of a board, including how many units are built.

- Calculate cost, stock on hand, shortfall and build readiness in the database itself.
- Use constraints and triggers so that invalid data (negative stock, a build finishing before it started) is rejected by the database.

2.2 Users

This is a single-user system. I use it in three roles:

Role	What the role does in the system
Designer	Imports BOM files from KiCad, corrects part details, adds new boards.
Purchaser	Adds suppliers, records price quotes, records parts that are bought.
Builder	Plans a build, checks whether stock is enough, marks the build as complete.

2.3 Scope

In scope	Out of scope
BOM import from KiCad CSV exports	Schematic and PCB layout data
Parts catalogue and categories	Placing orders with suppliers automatically
Supplier price quotes (with date)	Multiple users, logins and permissions
Stock ledger (purchases, adjustments)	Live price feeds from supplier APIs
Build planning and completion	Accounting, invoices and taxes
Cost, shortfall and readiness reports; Python front end	

3. Requirement Analysis

3.1 Source Data

The starting data is the real BOM export of uGameBox (AT89S52 microcontroller, 16x2 LCD, eight LEDs, nine push buttons, buzzer, SW-420 vibration sensor, L7805 regulator). A few lines from the file, unchanged:

Refs	Value	Footprint (shortened)	Qty
`R1-R8,R10`	1k	`R_Axial_DIN0207_L6.3mm...`	9
`C1,C6,C7`	100nF	`C_Disc_D5.0mm_W2.5mm_P5.00mm`	3
`D1-D8`	LED	`LED_D5.0mm`	8
`SW1`	BTN0	`SW_PUSH_6mm`	1
`SW9`	RESET	`SW_PUSH_6mm`	1
`U1`	AT89S52	`DIP-40_W15.24mm_Socket`	1

The file has 28 lines covering 49 placed components. I checked that the Qty of every line matches the number of designators it lists. The importer expands each line into one row per designator. The same importer can load the BOM of any other KiCad project, so the database is designed for many boards, not just one.

3.2 Functional Requirements

ID	Requirement
FR1	Import a KiCad BOM CSV: expand designator ranges such as "R1-R8,R10" into one placement each, and link each placement to a part.
FR2	Maintain a parts catalogue grouped into categories (resistor, capacitor, LED, IC and so on). Identical parts must not be duplicated.
FR3	Maintain boards (name, revision, source) and the list of placements on each board.
FR4	Record suppliers and dated price quotes for parts.
FR5	Record stock movements: purchases (linked to a supplier) and manual adjustments.
FR6	Plan a build of N units of a board and mark it complete.
FR7	Report: BOM per board, cost per board, stock on hand, shortfall for a build, parts shared across boards.
FR8	Reject invalid operations: negative stock, completing a build without enough stock, invalid dates, non-positive prices.

3.3 Entities and Attributes

Entity	Attributes	Meaning
CATEGORY	category_id, name	Type of part (Resistor, LED, ...)
PART	part_id, category_id, value_text, footprint, description	One distinct physical part that can be bought
BOARD	board_id, name, revision, source, created_on	A PCB design, e.g. uGameBox rev 1
PLACEMENT	placement_id, board_id, part_id, ref_designator, label	One component position on a board (R3, SW5, ...)
SUPPLIER	supplier_id, name, website	A shop or distributor
PRICE_QUOTE	quote_id, part_id, supplier_id, unit_price, currency, captured_at	Price seen for a part at a given time
BUILD	build_id, board_id, units, started_on, completed_on	One assembly run of a board
STOCK_MOVEMENT	movement_id, part_id, supplier_id, qty_delta, movement_type, moved_at	Parts coming in (purchase) or a correction (adjustment)

4. ER Diagram

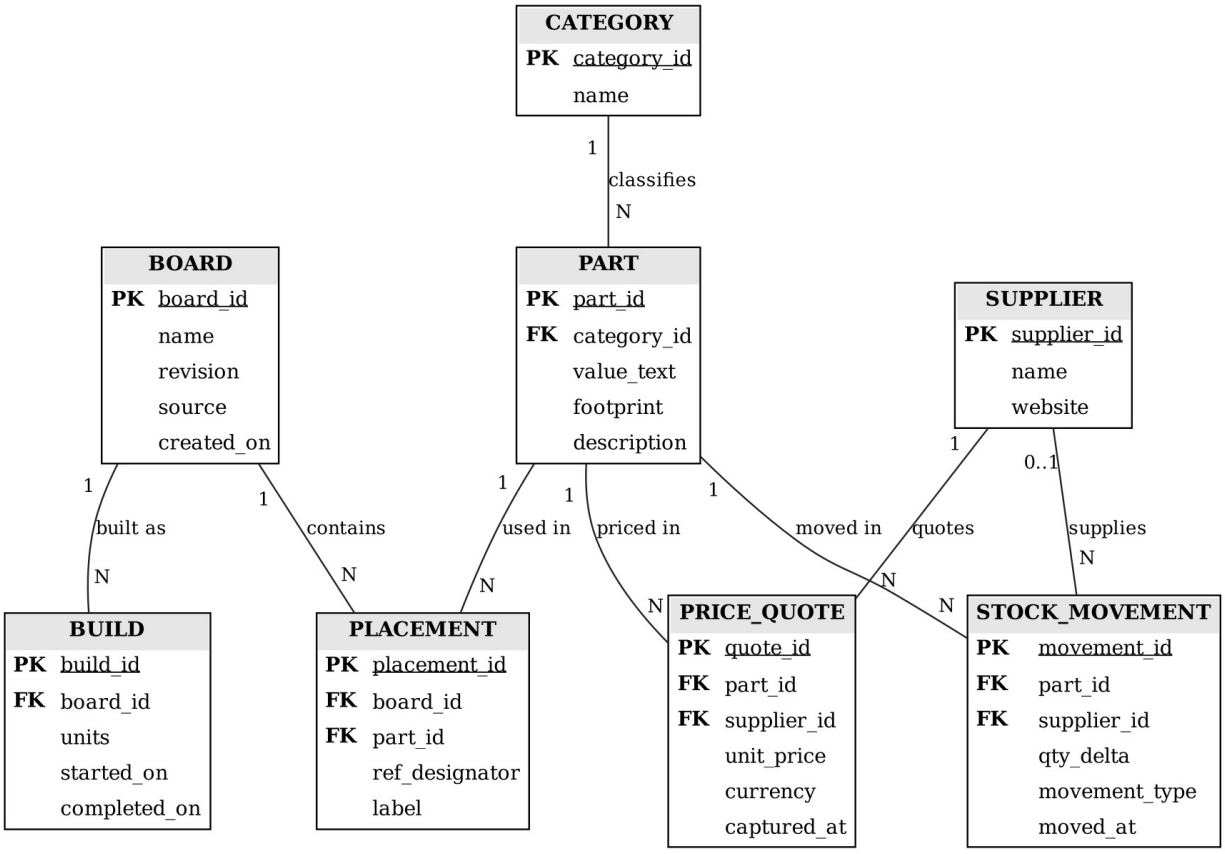


Figure 1. ER diagram of BOM-Track (PK = primary key, FK = foreign key)

4.1 Relationships and Cardinality

Relationship	Card.	Explanation
CATEGORY classifies PART	1 : N	Every part belongs to exactly one category.
PART used in PLACEMENT	1 : N	A part can be placed many times; a placement uses one part.
BOARD contains PLACEMENT	1 : N	A board has many placements; a placement is on one board.
BOARD and PART	M : N	Resolved through PLACEMENT, which also carries ref_designator and label.
BOARD built as BUILD	1 : N	The same board can be built many times.
PART priced in PRICE_QUOTE	1 : N	A part can have many quotes over time.
SUPPLIER quotes PRICE_QUOTE	1 : N	PART and SUPPLIER are M : N through PRICE_QUOTE.
PART moved in STOCK_MOVEMENT	1 : N	Each stock movement is for one part.
SUPPLIER supplies STOCK_MOVEMENT	0..1 : N	A purchase names a supplier; an adjustment does not.

Participation: every PART must have a CATEGORY and every PLACEMENT must have both a BOARD and a PART (total participation on the N side). A BOARD may exist with no BUILD yet, and a PART may exist with no quotes or stock movements (partial participation).

5. Relational Schema

*Primary keys are underlined, foreign keys are marked with *.*

CATEGORY (category_id, name **)**

PART (part_id, category_id*, value_text, footprint, description **)**

BOARD (board_id, name, revision, source, created_on **)**

PLACEMENT (placement_id, board_id*, part_id*, ref_designator, label **)**

SUPPLIER (supplier_id, name, website **)**

PRICE_QUOTE (quote_id, part_id*, supplier_id*, unit_price, currency, captured_at **)**

BUILD (build_id, board_id*, units, started_on, completed_on **)**

STOCK_MOVEMENT (movement_id, part_id*, supplier_id*, qty_delta, movement_type, moved_at **)**

5.1 Keys and Constraints

Table	Foreign keys	Other constraints
CATEGORY	none	name NOT NULL UNIQUE
PART	category_id -> CATEGORY	value_text, footprint NOT NULL; UNIQUE(category_id, value_text, footprint)
BOARD	none	name, revision NOT NULL; UNIQUE(name, revision); created_on DEFAULT CURRENT_DATE
PLACEMENT	board_id -> BOARD (ON DELETE CASCADE); part_id -> PART	ref_designator NOT NULL; UNIQUE(board_id, ref_designator)
SUPPLIER	none	name NOT NULL UNIQUE
PRICE_QUOTE	part_id -> PART; supplier_id -> SUPPLIER	CHECK (unit_price > 0); currency DEFAULT 'INR'; captured_at DEFAULT now()
BUILD	board_id -> BOARD	units DEFAULT 1, CHECK (units > 0); started_on NOT NULL; CHECK (completed_on >= started_on)
STOCK_MOVEMENT	part_id -> PART; supplier_id -> SUPPLIER (nullable)	CHECK (qty_delta <> 0); movement_type IN ('PURCHASE','ADJUSTMENT'); a PURCHASE must have qty_delta > 0 and a supplier

6. Design Notes

6.1 What is stored and what is calculated

The CSV mixes stored facts with numbers that can be worked out from them. In BOM-Track, anything that can be calculated is not stored:

Value	Calculated from
Quantity of a part on a board	COUNT of PLACEMENT rows for that board and part (replaces the CSV Qty column)
Stock on hand	SUM of qty_delta in STOCK_MOVEMENT, minus parts used by

	completed builds (units x placements)
Board cost	Quantity of each part x its latest PRICE_QUOTE, added up
Shortfall for a build	Parts needed for the build minus stock on hand, when positive
Build status	From the dates: no completed_on means in progress, otherwise completed

Because of this, each fact has one home. There is no stock column that could disagree with the movement history, and no status column that could disagree with the build dates.

6.2 Normalization

The CSV is not in 1NF because the Refs column holds a list. Splitting it into one PLACEMENT row per designator fixes this. Separating PART from PLACEMENT removes repeated part data: the nine push buttons become one PART row and nine PLACEMENT rows, and each placement keeps its own label (BTN0, RESET, ...).

Table	Functional dependencies	Normal form
PART	part_id -> category_id, value_text, footprint, description; (category_id, value_text, footprint) -> part_id	BCNF
PLACEMENT	placement_id -> all; (board_id, ref_designator) -> placement_id, part_id, label	BCNF
PRICE_QUOTE	quote_id -> part_id, supplier_id, unit_price, currency, captured_at	BCNF
STOCK_MOVEMENT	movement_id -> part_id, supplier_id, qty_delta, movement_type, moved_at	BCNF
CATEGORY, BOARD, SUPPLIER, BUILD	Surrogate key -> all attributes; natural keys are declared UNIQUE	BCNF

In every table, each determinant is a candidate key, so there are no partial or transitive dependencies.

6.3 Plan for Review II and III

The schema already has a place for each component the guidelines ask for: views for the reports above; functions for stock on hand and board cost; procedures to record a purchase and complete a build; a cursor that walks through a build's shortfall to produce a shopping list; and triggers that stop stock from going negative and stop a build from being completed without enough parts. The Python front end will call these directly.